plan

Implementation Plan: Chapter Hierarchy

Branch: `003-chapter-hierarchy` | **Date**: 2026-09-05 | **Spec**: spec.md

Input: Feature specification from `specs/003-chapter-hierarchy/spec.md`

Summary

`workshop/chapters/` holds `01`, `02` and `02.01`. The last of those is the first sub-chapter this project has ever had, and everything downstream of `chapters/` was written against a flat list.

The identity decision is small and mostly already true: the canonical chapter id is the dotted, zero-padded numeric path, and it is simultaneously the directory name, the URL segment, the registry `scope`, the `chapter_slug` and the `chapter-` suffix, with no translation anywhere. **Four of the five layers a dotted id must survive already survive it** — `SafeSlug` permits `.`, `ChapterDir` resolves `chapter-02.01`, the route wildcard is single-segment, and `encodeURIComponent` leaves `.` alone. The fifth, the search catalogue's filename-shape detector, was measured and does not misclassify `.01`.

What is *not* already true is derivation. `ordinalOf("02.01")` returns `2`, which is what `ordinalOf("02")` returns, and `chapterTitle` renders both as *"Chapter 2"*. That is the visible half. The invisible half is larger: a redaction reviewer that matched scopes by suffix, two extraction stages that skipped a non-matching path with `continue`, six embedded manifest rows that name `chapter-01`, and an ingest script whose

transcript default is chapter 01's transcript — so omitting one argument ingests **the wrong chapter's content under the right chapter's scope**.

This plan fixes the invisible defects before the visible ones, and it fixes none of them before the specification decision they all rest on.

Technical Context

Language/Version: Go 1.26.2 for the backend, matching `workshop/platform/backend/go.mod`. TypeScript with Angular 19.2 standalone components for the front end. Python 3.14.6 and Bash for the pipeline. No new language and no new runtime.

Primary Dependencies: unchanged. Nothing is added. The identity library at `submodules/passage` is consumed as it already is — **chapter ids are not minted identifiers and this feature does not make them one**; they are structural names under an operator's control, and D3 in the specification records why that is deliberate.

Storage: unchanged in kind and in schema. `workshop/curriculum/passages.jsonl` remains the source of truth; its `scope` field already holds a chapter id as an opaque string and needs no change to hold a dotted one. **No column is added anywhere** — the hierarchy is derived (D3).

Testing: Go's built-in testing for the backend; Karma and Jasmine for the front end; the platform's existing gate scripts and route manifest for behavioural assertions against the live binary. Every new gate owes a paired mutation proof and a three-valued exit.

Target Platform: Linux, single machine, local and internal only. Unchanged.

Project Type: an in-place correction across an existing web application and its offline pipeline. There is no new component.

Performance Goals: none of its own. The existing latency thresholds are not renegotiated by this change, and nothing here is on a hot path — the hierarchy is a string operation over a list whose measured length is 3.

Constraints: no new routes (FR-026); no stored hierarchy field (FR-007); no code path branching on depth (FR-032); no server-side CI; the flat `chapters` array is a live consumer contract.

Premises checked before they were relied on

Each was assumed by the design and each was measured on 2026-09-05. **Five held and one was inverted**, and the inverted one is recorded as a correction rather than quietly dropped.

1. HELD — the filesystem layer already accepts a dotted id. `SafeSlug` (`pkg/curriculum/curriculum.go:205`) is an allowlist of `a-z A-Z 0-9 - _ .` that rejects `/` and `\` at `:209`; `ChapterDir` (`:187`) tries `chapter-<slug>` then `<slug>`. *Consequence for design*: the `chapters/02/01` alternative would have required **removing** the `/` rejection — a path-traversal boundary — to buy a naming preference. That is the strongest single argument for the dotted form and it is a measurement, not a preference.

2. HELD — the routing layer already accepts it. `GET /api/chapters/{chapter}` and five siblings are registered at `cmd/workshop-server/main.go:547`, `:817–:819`, `:841`, `:843`. Go's `{name}` wildcard matches exactly one path segment and treats `.` as an ordinary character. *Consequence*: no route pattern changes, and the `02/01` alternative would not have matched the registered patterns at all.

3. HELD — the client layer already accepts it. `encodeURIComponent` at `core/api.ts:107`, `:122`, `:155` leaves `.` unescaped; it is an unreserved character. *Consequence*: no client encoding change.

4. HELD — the ordering is already implemented correctly. `main.go:2126` sorts on `out[i].ID < out[j].ID`, which is byte-lexicographic. Run against `01 02 02.01 02.01.01 02.09 02.10 03 10` it produces exactly the required order. *Consequence for design*: **the correct action on the sorting code is to change nothing**, and to put the assertion on the ordering instead — a gate the code must satisfy rather than a rewrite that might satisfy it today.

5. HELD — the leak detector does not fire on a dotted id. `HasSourceFilenameShape` (`pkg/search/catalog.go:368`) compiles its pattern from a **closed** extension list (`:320–:323`) plus one archive-chunk alternative; the list holds no numeric-only entry, so `.01` is not read as an extension. *Consequence*: no allowance is needed, and no exemption is added. This one was checked precisely because a false positive here would have been a content-boundary alarm on every sub-chapter id.

6. INVERTED — the ordinal problem is not an implementation defect. The design's first framing was that `ordinalOf` is a bug to be fixed in `internal/api/chapters.go`. It is not: it is the faithful

implementation of `specs/001-workshop-curriculum-platform/data-model.md:45`, which types `Chapter.ordinal` as `int`. Under that type there is no correct implementation for `02.01`. *Consequence for the plan*: **Phase 0 is a specification decision, not code**, and it blocks everything. A second contradiction surfaced with it — `specs/001-workshop-curriculum-platform/contracts/http-api.md:60` says the zero-padded ordinal is *"not accepted as a path key"* while `route-manifest.tsv:72` declares `GET /api/chapters/01`, which is exactly that. Both are inherited defects, neither was introduced here, and neither may be inherited silently.

Constitution Check

GATE: must pass before Phase 0 research. Re-check after Phase 1 design.

Principle	Status	Basis
Evidence-Based Claims	**PASS**	Every figure in the specification's starting-state table names the file and line, or the command, that re-derives it. The sort order was produced by running `sort`, not by reasoning about byte values. One premise was **inverted by measurement** and is recorded as such.
Honest Instruments	**PASS**	FR-004 forbids a silent skip and requires an `unclassified` report with a reason. FR-028 requires the filter echo on **every** status including could-not-determine, so a degraded backend can never read as an empty branch.

Principle	Status	Basis
		FR-029 refuses a `404` for a well-formed request about a branch that does not exist.
Governance Fidelity	**PASS**	No carrier changes, no submodule added, no governance surface touched. FR-033 requires the route manifest to move in the same change as the behaviour.
Isolation by Default	**PASS**	Every criterion guarding a silent failure names its mutation: SC-001, SC-002, SC-003, SC-005, SC-006, SC-007, SC-008, SC-010, SC-011, SC-013, SC-014, SC-017 and SC-019. SC-009 goes further and measures the **blindness of a test** rather than the behaviour of the code.
Comprehensive Documentation	**PASS**	FR-024 requires `derivation.ordinal` to keep describing what the code actually does; the endpoint's existing honesty mechanism is preserved rather than left stale.
Environment Adaptability	**PASS**	FR-019 removes a frozen absolute path; FR-016 removes a

Principle	Status	Basis
		frozen transcript default; FR-021 removes six frozen manifest rows; FR-035 requires every gate to take its chapter from the live tree, as one gate already does.
Quality Over Speed	**PASS**	The grammar, the two-scope redaction fixture and the ordering gate are marked test-first below, chosen where a defect is **silent**.
§11.4.156 — no server-side CI	**PASS**	Nothing here adds a workflow file anywhere in the fleet.

Gate result (initial): PROCEED to Phase 0 only. The clarification recorded in the specification blocks Phase 1 and everything after it. Phase 0 is the work of settling it.

What the gate does *not* certify

This table records that the **design** satisfies the principles. It is not a claim that the platform's gates pass today — none was run for this plan, and two that write into tracked evidence files were deliberately not invoked. It is also not a claim that the open clarification has a safe default. It does not: all three options for the `ordinal` type change something a consumer can see, and the *appearance* of a safe default — keep `int`, leave sub-chapters without an ordinal — is the option that permanently makes them second-class.

Phase ordering, and the rule behind it

Fix what hides first.

The three defects already repaired in `266f443` share a shape: none of them failed. The redaction reviewer exited `0` and produced a coverage record that overstated itself; the two extraction stages exited `0` and produced nothing. A defect that announces itself gets fixed by whoever trips over it. A defect that returns `200` and exits `0` gets fixed only if someone goes looking, and it does damage the whole time nobody does.

So the phases are ordered by **how loudly each defect fails**, quietest first — after two phases that are ordered by necessity rather than by noise:

Phase	What	Why here
0	The specification decision — `ordinal` type, path-key contradiction	**Blocks everything.** No implementation of `ordinal_path` is correct until the type it replaces is resolved, and shipping one first would make the decision retroactively.
1	Foundational — the grammar, the validator, the derivation, the ordering gate	**Enabling.** Nothing can be *asserted* about `02.01` before there is a definition of what a chapter id is. This phase writes no fix; it writes the thing every fix is checked against.
2	The redaction defect's owed test	**Quietest.** The fix has landed; the test has not. It is harmless with one scope and wrong with two, so the code currently passes for a reason that has nothing to do with being correct. A one-scope fixture cannot see it

Phase	What	Why here
		— SC-009 measures exactly that.
3	Silent data loss — ingest defaults, transcript defaults, embedded manifest, frozen builders	**Next quietest.** Each of these produces plausible output. The ingest default is the worst of them: it writes **chapter 01's transcript under another chapter's scope**, and every downstream artifact then agrees with itself.
4	Visible defects — `ordinalOf`, `chapterTitle`, the front end's two shapes, the route manifest, `calibrate.sh`	**Loud, or nearly.** A collision in a title is visible to anyone who looks at two chapters. The front end's `null`-versus-`-2` disagreement is visible to anyone who compares two views. These get fixed anyway; they do not need to be first.
5	The API surface — `hierarchy`, the filters, the orphan shape	**Depends on 1 and 4.** It serves what those derive.
6	Gates, manifest, documentation, closure	**Last, by construction.** A gate written before the behaviour it guards is a gate written against an intention.

Phase 2 before Phase 3 is a judgement, and it is stated rather than implied. Both are silent. The redaction defect is put first because its output is a **disclosure-control record** — a document whose whole

purpose is to state what was covered — and a coverage claim that is wrong is worse than a content pipeline that produced nothing, which at least leaves an absence somebody notices.

Project Structure

Documentation (this feature)

```
specs/003-chapter-hierarchy/
├── spec.md                        # Feature specification
├── plan.md                        # This file
├── research.md                    # Measured evidence and the
rejected alternatives
├── data-model.md                  # The Chapter entity, the
derived view, invariants H1–H7
├── contracts/
│   └── http-api.md                # The two changed response
bodies
└── tasks.md                       # Task breakdown, T001–T037
```

Source Code

Paths by responsibility. Every one of these already exists — **this feature creates no new package, no new module and no new component.**

```
workshop/platform/backend/
├── pkg/curriculum/curriculum.go        # EXISTING and ALREADY
CORRECT — SafeSlug:205 permits `.`,
│                                       #   ChapterDir:187
resolves `chapter-02.01`. Gains an
│                                       #   ASSERTION, not a
change.
├── pkg/chapterid/                      # NEW, and the only new
package — the grammar, the
│                                       #   validator and the five
derivations. Depends on nothing
│                                       #   but the standard
library, which is what makes it
│                                       #   testable with no tree
present.
├── internal/api/chapters.go            # EXISTING — ordinalOf:736
and chapterTitle:755 replaced by
│                                       #   calls into pkg/
chapterid; derivation:141 rewritten.
├── pkg/search/suggest-sources.json      # EXISTING — six
chapter-01 rows derived instead
├── pkg/search/catalog.go               # EXISTING — the //
```

```
go:embed at :70–71 and the manifest
│                                    #   loader; measured safe
at :320–354, gains an assertion
├── cmd/workshop-server/main.go      # EXISTING —
listChapters:2094 gains the unclassified
│                                    #   report; the comparator
at :2126 MUST NOT CHANGE
└── cmd/workshop-redact/main.go      # EXISTING — :616 already
equality; gains the two-scope test

workshop/pipeline/
├── extract/meeting_notes.py         # EXISTING — :109 already
widened; gains both-direction proof
├── extract/author.py                # EXISTING — :159 already
widened; gains both-direction proof
├── build_transcript.py              # EXISTING — :143–144
frozen defaults removed
└── calibrate.sh                     # EXISTING — :36 frozen
absolute path resolved from a chapter

workshop/scripts/ingest.sh           # EXISTING — :46 chapter
default, :93 transcript default
workshop/curriculum/chapter-01/knowledge/build.py
                                     # EXISTING — :363, :435
deep links and :478, :564, :582,
                                     #   :592 slug literals
derived as :415 already does

workshop/platform/frontend/src/app/core/
├── models.ts                        # EXISTING — :301 and :357
reconciled
└── api.ts                           # EXISTING — :88 reads the
array flat; must keep working

workshop/platform/gates/
├── route-manifest.tsv               # EXISTING — :64 #subst,
rows :72–:76, :85, :157–:160
├── verify-chapter-hierarchy.sh      # NEW — the ordering,
grammar and no-special-case gate
├── prove-chapter-hierarchy.sh       # NEW — its paired
mutation proof
└── verify-absence-honesty.sh        # EXISTING — :127–134 is
the precedent every new gate copies
```

Structure Decision: the one new package is `pkg/chapterid`, not `internal/`. It knows about a string grammar and nothing else — no registry, no filesystem, no HTTP — which is what lets it be tested exhaustively against a table of inputs with no tree present, and what lets the pipeline's own checks assert against the same definition rather than a second one. `internal/` would foreclose that by language rule.

Everything else is an edit to a file that already exists. **A feature whose implementation adds one small package and changes fourteen lines across nine files is a feature whose risk is concentrated in the decisions, not in the code**, which is why Phase 0 is where the weight is.

Execution Strategy

Test-first components

Marked `[TDD]` in the task breakdown. Chosen where a defect would be **silent** rather than loud.

- [ ] **The id grammar and its validator.** The one input it exists to catch — `02.2` — sorts incorrectly and looks fine. No sorting test written with well-formed input can see it, so the rejection is written before the acceptance.
- [ ] **The two-scope redaction fixture.** The defect is invisible to a one-scope fixture by construction, and SC-009 requires that blindness to be *measured*, not assumed. Writing the fixture after the assertion would invite a fixture shaped to pass.
- [ ] **The three-order agreement.** API, glob and filesystem order must be the same order. Nothing asserts it today and the code that satisfies it is one refactor from not doing so.
- [ ] **The derivations.** `parent_id`, `depth`, `ancestor_ids`, `child_ids`, `ordinal_path` — each a pure string operation, each with a table of inputs including the root and leaf cases that FR-008 requires to fall out rather than be special-cased.
- [ ] **The ingest refusal.** A test that ingestion *refuses* is the only thing that distinguishes it from ingestion that quietly used another chapter's transcript, because both write records and both exit successfully today.
- [ ] **The both-direction pattern proofs.** A widening that stopped matching the old form would pass a test asserting only the new form. Both directions, or the proof is half a proof.

Parallel work

Marked `[P]`. These share no files.

- [ ] The pipeline defaults (`ingest.sh`, `build_transcript.py`, `calibrate.sh`) and the backend derivation — different languages, different directories.
- [ ] The front-end shape reconciliation and the suggest-source derivation.
- [ ] The route manifest rows and the documentation.

Human checkpoints

The agent stops and waits for explicit approval at each.

1. **Before anything in Phase 1** — the `ordinal` type and the path-key contradiction are operator decisions about two documents feature 001 already published. They are not implementation choices and must not be settled by writing code.
2. **After the grammar lands** — every later assertion is written against it, and a grammar changed after adoption invalidates all of them.
3. **After Phase 3** — the silent-loss fixes change what the pipeline writes. The diff is reviewed before any chapter is re-ingested.
4. **Before the API change is declared complete** — the flat-array contract with `api.ts:88` is checked against a running client, not against the schema.

Review gates

Marked `[REVIEW]`.

- [ ] **The Phase 0 decision record.** Both amendments to feature 001's artifacts are written down, with their reasoning, before any code depends on them.
- [ ] **The grammar**, before anything consumes it.
- [ ] **The two-scope fixture**, because it is the only artifact in this feature that proves a disclosure-control record means what it says.

- [ ] **The route manifest diff**, in the same change as the behaviour it declares.

Complexity Tracking

Violation	Why needed	Simpler alternative rejected because
A new package (`pkg/chapterid`) for what is arguably five one-line string functions	The grammar is consumed by the backend, the pipeline and the gates. One definition, three consumers.	Inlining the parsing at each call site is simpler per site and guarantees three subtly different definitions of what a chapter id is — which is how `ordinalOf` came to exist in the first place.
A `hierarchy` object on every row rather than a nested array	The client needs ancestry; a live consumer reads the array flat (`api.ts:88`).	Nesting is smaller on the wire and is a breaking change to a working client, forcing a tree walk to find a chapter that is currently indexable.
`ordinal_path` replacing `ordinal`, changing a published wire field	`02.01` has no `int` ordinal, and the two collide under one.	Keeping `int` and giving sub-chapters no ordinal is a smaller diff and makes every sub-chapter permanently unorderable and unlabelled — it does not solve the problem, it names it and moves on.
A criterion (SC-009) that asserts a test **stays green** under a mutation	It is the only way to prove a one-scope fixture cannot see the suffix defect.	Asserting only that the two-scope fixture goes red proves the fixture works; it does not prove the

Violation	Why needed	Simpler alternative rejected because
		cheaper fixture somebody will write instead does not.